Domain 5: Context Management & Reliability — Practice-Based Self-Questioning

---

1. Long session approaching the window limit, must continue same task

- Self-question: Does periodic compaction (summarize completed work/decisions, then continue on summary + recent messages) keep the task going — rather than letting the API auto-truncate, stopping at 80%, or switching to a smaller model?
- Key distinction: compaction vs. relying on auto-truncation of oldest messages vs. hard stop at 80% vs. smaller model — which preserves task continuity while freeing space?

2. Plausible-but-nonexistent policy number (no verification tool)

- Self-question: Is grounding the agent with a tool/retrieval system that looks up real policy numbers the reliable fix — rather than "double-check your facts," a bigger window, or lower temperature?
- Key distinction: retrieval grounding in verified data vs. self-check instruction vs. larger window vs. temperature — which prevents relying on the model's internal (unverifiable) knowledge?

3. Tool results >50K tokens degrade the next few responses

- Self-question: Is the cause that large tool results crowd out earlier context, with the fix being a filtered/summarized result containing only task-relevant fields — rather than a hard size limit, "no fix exists," or regenerating IDs?
- Key distinction: context crowding + filtered results vs. hard 50K rejection vs. "expected, unfixable" vs. tool_use_id mismatch — which correctly diagnoses context rot and trims at the source?

4. Multi-day customer thread; full replay is costly and mostly resolved chatter

- Self-question: Does storing a structured case summary (issue, key facts, actions, status) + loading it plus the most recent few turns balance cost and continuity — rather than starting fresh, full replay, or re-reading the whole transcript?
- Key distinction: structured summary + recent turns vs. discard-and-restate vs. full transcript every time vs. re-read from scratch — which keeps continuity without paying for resolved history?

5. Budget stated in turn 2 violated by a recommendation in turn 40 (all in window)

- Self-question: Even with the full history in the window, does early information get diluted — with the fix being to periodically re-surface active constraints in a structured block near the top of context or just before the current turn?
- Key distinction: re-surface constraints (lost-in-the-middle mitigation) vs. "API bug" vs. "recency-only attention makes it permanently inaccessible" vs. restart every 10 turns — why does in-window presence not guarantee recall?

6. Repeated lookup_order results now dominate context

- Self-question: Before more lookups, should you extract only return-relevant fields (items, purchase date, return window, status) from each existing response — rather than proceeding unchanged,

prose-summarizing, or a vector DB?
- Key distinction: field-level trimming of existing tool output vs. proceed as-is vs. natural-language summary vs. vector database — which reclaims budget while keeping the exact needed values?

7. Agent re-asks for the customer's name at verification step 3

- Self-question: Is the most likely cause that conversation history isn't being passed in subsequent API requests — rather than a missing "remember" instruction, a 2-turn memory limit, or the tool clearing internal state?
- Key distinction: history not passed (stateless API reality) vs. missing prompt instruction vs. "default 2-turn memory" (not real) vs. tool clearing state — which reflects how statelessness actually works?

8. Resume session; 3 of 12 previously-read files changed overnight

- Self-question: Should you resume and inform the agent which specific files changed for targeted re-analysis — rather than resuming silently, re-reading all 12, or starting fresh?
- Key distinction: resume + inform about the 3 changes vs. resume silently (stale assumptions) vs. re-read all 12 (wasteful) vs. start fresh (loses the 9 unchanged) — which balances efficiency and accuracy?

9. Find all callers of a function exposed under renamed wrappers

- Self-question: Should you read the library and wrapper modules to collect all exposed names, then Grep each name — rather than grepping only the original name, checking docs, or grepping imports then reading each file?
- Key distinction: read-for-all-names then Grep-each vs. Grep original name only (misses renames) vs. documentation search vs. import-grep + read-each — which reliably catches renamed re-exports?

10. Resume a named 2-hour session after working on other codebases

- Self-question: Should you use `--resume auth-deep-dive` (by name) — rather than `--session-id` with a UUID, starting fresh, or `--continue`?
- Key distinction: `--resume <name>` vs. `--session-id <UUID>` (works but needs the transcript file) vs. start fresh vs. `--continue` (picks most recent, possibly the wrong codebase) — which loads the *specific* named session?

11. 45-file payment module; accuracy degrading after 8 files, work incomplete

- Self-question: Should you spawn subagents for specific questions (find test files, trace refund flow) while the main agent coordinates and keeps high-level understanding — rather than /clear + re-read, a summary-only reference, or Grep-only exploration?
- Key distinction: subagent delegation for bounded investigations vs. /clear + scratchpad re-read vs. summary-as-sole-reference vs. Grep-only — which isolates verbose work while preserving the main session's understanding?

12. Add tests to a 200-file legacy codebase, no priority specified

- Self-question: Should the agent use Glob/Grep to map structure, identify heavily-coupled modules, build a prioritized high-impact plan, and revise as dependencies emerge — rather than a fixed

directory schedule, alphabetical start, or reading all 200 first?
- Key distinction: map + prioritize + adapt vs. fixed directory-based schedule vs. alphabetical organic discovery vs. read-all-200-first (wastes context) — which decomposes an open-ended task by impact?

13. Explore two refactoring approaches in depth before deciding

- Self-question: Should you use `fork_session` to branch from yesterday's analysis, exploring one approach per fork — rather than sequential in one thread, resume-then-new-session, or two fresh sessions?
- Key distinction: fork_session (preserves shared context, isolates branches) vs. sequential in same thread (approaches contaminate each other) vs. resume + new session (recreate context) vs. two fresh sessions (lose context) — which explores independently from a shared baseline?

14. Pivoting from rendering to physics; agent losing specificity on earlier classes

- Self-question: Should you summarize key rendering findings, then spawn a sub-agent for physics with that summary in its initial context — rather than continuing with targeted prompts, spawning without a summary, or /clear?
- Key distinction: summarize + spawn-with-summary vs. continue in degrading context vs. spawn-then-manually-synthesize vs. /clear (destroys accumulated knowledge) — which preserves prior findings while giving new work a fresh window?

15. Resume a 47-file exploration; 2 utility functions renamed while away

- Self-question: Should you resume the subagent from its transcript and inform it about the renamed functions — rather than resuming silently, a fresh subagent with the transcript, or a fresh subagent with a summary?
- Key distinction: resume + inform of renames vs. resume silently (stale) vs. fresh + full transcript vs. fresh + summary (loses detail) — which keeps accumulated context while correcting the specific staleness?

16. Building understanding of a 15-file, 8,000-line caching layer

- Self-question: Should you analyze imports/class hierarchies to find the base cache class, read that for the interface, then trace specific implementations — rather than reading all 15 sequentially, grep-and-read-line-ranges, or largest-file-first?
- Key distinction: top-down from the architectural entry point vs. sequential full reads (fills window) vs. grep-then-narrow-ranges vs. size-based ordering — which builds structural understanding while managing context?

17. Trace intermittent 500 errors through 4 layers, components unknown

- Self-question: Should the agent dynamically generate investigation subtasks based on what it discovers at each step — rather than parallel all-four-layers, a comprehensive upfront plan, or a fixed sequence regardless of findings?
- Key distinction: adaptive/dynamic decomposition vs. parallel investigation of all layers vs. exhaustive upfront mapping vs. fixed sequence — which fits an error path that's unknown until discovered?

18. Independently develop two testing strategies from a 23-file analysis

- Self-question: Should you resume the analysis session with `fork_session`, creating a branch per strategy — rather than exporting findings to a file for two new sessions, two fresh re-reading sessions, or sequential in one session?
- Key distinction: fork_session (shared analysis, isolated branches) vs. export-to-file + two new sessions vs. two fresh sessions re-reading vs. sequential (contamination) — which preserves the analysis while keeping the two strategies isolated?

19. Three separate issues in one session, approaching context limits

- Self-question: Should you extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer — rather than MCP re-fetch on demand, a 30-turn sliding window, or narrative summarization?
- Key distinction: structured issue data in a separate layer vs. on-demand MCP re-fetch vs. sliding window (drops early issues) vs. narrative summary (loses exact values) — which keeps durable facts for *all* issues available?

20. Returning customer; agent cites stale PENDING tool results even after fresh calls

- Self-question: Should you start a new session and inject a structured summary (issue, actions, status) then make fresh tool calls before engaging — rather than resuming with auto-re-call-all-tools, filtering old tool_results, or a "prefer most recent" prompt instruction?
- Key distinction: new session + structured summary + fresh calls vs. resume + auto-recall-all vs. resume + filter tool_results vs. "prefer most recent" instruction (advisory, unreliable) — which reliably prevents the agent from anchoring on stale data?